

Sequence analysis

SOAP: short oligonucleotide alignment programRuiqiang Li^{1,2}, Yingrui Li¹, Karsten Kristiansen² and Jun Wang^{1,2,*}¹Beijing Genomics Institute at Shenzhen, Shenzhen 518083, China and ²Department of Biochemistry and Molecular Biology, University of Southern Denmark, Odense M, DK-5230, Denmark

Received on November 10, 2007; revised on December 20, 2007; accepted on January 14, 2008

Advance Access publication January 28, 2008

Associate Editor: Keith Crandall

ABSTRACT

Summary: We have developed a program SOAP for efficient gapped and ungapped alignment of short oligonucleotides onto reference sequences. The program is designed to handle the huge amounts of short reads generated by parallel sequencing using the new generation Illumina-Solexa sequencing technology. SOAP is compatible with numerous applications, including single-read or pair-end resequencing, small RNA discovery and mRNA tag sequence mapping. SOAP is a command-driven program, which supports multi-threaded parallel computing, and has a batch module for multiple query sets.

Availability: <http://soap.genomics.org.cn>

Contact: soap@genomics.org.cn

The new DNA sequencing technologies, which have been developed and implemented recently, have significantly improved throughput and dramatically reduced the cost compared with the capillary-based electrophoresis systems (Shendure *et al.*, 2004). In a single experiment using one instrument, the Illumina-Solexa system using sequencing-by-synthesis (SBS) can determine up to 40 million sequences of up to 50 bases in length, whereas the ABI-SOLiD system using ligation technology allows determination of 3 Gb mappable data. The ultra high throughput and short read length make these technologies particularly suitable for large-scale resequencing of large cohorts of individuals with known reference for studies of genetic variations (Bentley, 2006). Traditional sequence alignment software like blast (Altschul *et al.*, 1997) and blat (Kent, 2002) are unable to cope efficiently with the huge amount of reads generated in such applications, while SSAHA (Ning *et al.*, 2001) is optimized to find long alignments and fails in practice on most short queries. To our knowledge, there have been several programs developed or under developing to match the new sequencing technologies. ELAND, an alignment tool integrated in Illumina-Solexa data processing package, can do ungapped alignment for reads with size up to 32 bp (Cox, unpublished). Maq is another program for ungapped alignment, which implemented sophisticated probability models to measure alignment quality of each read using sequence quality information (Li, unpublished). Here, we present a new program SOAP, which can do both ungapped and gapped alignment, and has special modules for alignment of pair-end, small RNA and mRNA tag sequences.

SOAP will allow either a certain number of mismatches or one continuous gap for aligning a read onto the reference sequence. The best hit of each read which has minimal number of mismatches or smaller gap will be reported. For multiple equal-best hits, the user can instruct the program to report all, or randomly report one, or disregard all of them. Since the typical read length is 25–50 bp, hits with too many mismatches are unreliable which are hard to distinguish with random matches. By default, the program will allow at most two mismatches. Between two haplotype genome sequences, occurrence of single nucleotide polymorphism is much higher than that of small insertions or deletions, so ungapped hits have precedence over gapped hits. For gapped alignment only one continuous gap with a size ranging from 1 to 3 bp is accepted, while no mismatches are permitted in the flanking regions to avoid ambiguous gaps. The gap could be either insertion or deletion in the query or the reference sequence. As the intrinsic character of the sequencing technology, errors will accumulate during the sequencing process. Reads always exhibit a much higher number of sequencing errors at the 3'-end, which sometimes make them unalignable to the reference sequences. To deal with the problem, SOAP can iteratively trim several basepairs at the 3'-end and redo the alignment, until hits are detected or the remaining sequence is too short for specific alignment.

Pair-end sequencing means to sequence both ends of a DNA fragment. So the two reads belonging to a pair will always have the settled relative orientation and approximate distance between each other on the genome. The technology can significantly improve the accuracy of resequencing mapping, and is a powerful method for detection of structural variants including copy number variations (CNVs), rearrangements, inversions and etc. SOAP is able to align a pair of reads simultaneously. A pair will be aligned when two reads are mapped with the right orientation relationship and proper distance. Similar filter as single-read alignment, a certain number of mismatches are allowed in one or both reads of the pair. For gapped alignment, gap is only permitted on one read, and the other end should match exactly.

Apart from genome resequencing, The high throughput sequencing technology lends itself to numerous applications. For some applications (ex. ChIP-Seq), the data analysis process is essentially identical to that of resequencing. Additionally, SOAP provides special modules for small RNA discovery and mRNA tag profiling analysis. Small RNAs have a size between

*To whom correspondence should be addressed.

18 to 26 bp. According to the experimental protocol, the 3'-end of RNA sequence will be flanked by adapter sequences. SOAP will filter adapter sequence, and then align the remaining candidate small RNA to the reference sequence. A small RNA will be annotated if an adapter sequence is detected and the insert sequence match well with the reference sequence. Considering sequencing errors, one or two mismatches can be allowed inside either the adapter or the candidate RNA region according to user settings. On mRNA tag sequencing, there are two types of restriction enzyme digestion: (i) DpnII, which will specially recognize the site 'GATC' and cuts a 16 bp tag after the site; (ii) NlaIII, which exclusively recognizes the site 'CATG' and cuts 17 bp downstream. SOAP checks and trims off the 3'-end adapter sequence according to the enzyme type. Aligned hits should contain the enzyme site, and have at most one mismatch in the tag region.

SOAP uses seed and hash look-up table algorithm to accelerate alignment. Both reads and the reference sequences are converted to numeric data type using 2-bits-per-base encoding. A read will do exclusive-OR comparison with the reference sequence. Then the value is used as suffix to check the look-up table to know how many bases are different. In order to have a tradeoff between memory usage and efficiency, SOAP uses unsigned 3-bytes data type as the table element. To admit two mismatches, a read is splitted into four fragments, the two mismatches can exist in at most two of the fragments on the same time, then if we try all six combinations of the two fragments as seed, we can however catch all hits with two mismatches (the algorithm is the same as Eland and Maq). Since mismatches are not allowed in gapped hits, SOAP used the enumeration algorithm which tries to insert a continuous gap or delete a fragment at each possible position in a read. The algorithm outputs the identical alignments as that of dynamic programming while runs much faster. Not unlike Eland and Maq which load read sequences into memory and build seed index tables for reads, SOAP loads reference sequences into memory as an unsigned 3-bytes array and builds the seed index tables for all the reference sequences. Then for each read, create seeds and search the corresponding index table for candidate hits, perform alignment and report the results. The RAM required for storing the reference sequences and seed index tables can be calculated as:

$$RAM = \frac{L}{3} + (4 * 3 + 8 * 6) * 4^S + (4 + 1) * 3 * \frac{L}{4} + 4 * 2^{24}$$

where L is the total length of the reference sequences; S is seed size. For small reference like yeast, $L = 12$ Mb and selected seed size $S = 10$ bp, about 200 Mb RAM is needed; but for the whole human genome, $L = 3$ Gb and a selected seed size $S = 12$ bp, about 14 Gb RAM in total will be needed.

Evaluated on a real dataset containing 9914527 Illumina-Solexa single-end resequencing reads (length 32 bp), which were generated from a 5 Mb human genome region, SOAP was almost 300 (gapped) to 1200 (ungapped) times faster than blastn, while having better sensitivity (Table 1). The iterative feature of SOAP improved sensitivity. And gapped alignment can further identify hits accommodating small indels which compose only a small fraction of all hits but are a very important class of mutation. Since SOAP loads reference sequences into memory, while Eland and Maq load reads, the memory usage varies in different datasets.

Table 1. Comparison of performance and sensitivity among short oligonucleotide alignment programs

Program	Time consumed (s)	Reads aligned (%)
blastn (-F F -W 11)	165 780	85.47
blastn (-F F -W 15)	150 660	84.66
Blat (-tileSize = 8)	22 032	85.07
Eland	166	88.53
Maq	458	88.39
Soap	134	88.46
Soap iterative	161	90.9
Soap iterative + gapped	486	91.15

We used a query dataset containing 9914527 single-end reads (length 32 bp) generated by Illumina-Solexa Genome Analyzer. The DNA sample is a mixture of long PCR products of a 5 Mb human genome region. For blastn we tried 11(better sensitivity) and 15(faster) bp word size, and disabled DUST masking of low-complexity sequence (-F F). For blat, tileSize parameter was set at 8. SOAP used 12 bp seed, SOAP iterative will iteratively trim off 2 bp at 3'-end of read and redo alignment until hits were detected or the remaining sequence is shorter than 27 bp. Sensitivity is calculated under the same threshold by allowing at most 2 mismatches. SOAP gapped will allow one continuous insertion or deletion with size between 1 to 3 bp. After checking sequencing quality, we found the remaining unmappable reads are in low sequencing quality.

SOAP accepts FASTA format for reference, and either FASTA or FASTQ format for query reads. It's a command-driven program, which employs single command line model and batch computing model. On batch computing model, the reference sequences and hash index tables will reside in the memory and alignment procedure can be performed for multiple query datasets in a order. This model avoids I/O and time wasted on loading reference and creating hash tables multiple times, and is also suitable for real-time web service. SOAP is written in standard C++ language and runs well on Macintosh or any 64-bit Linux/Unix systems. It supports multithreaded parallel computing.

ACKNOWLEDGEMENTS

We are indebted to Wei Fan, Qibin Li, Xiaodong Fang, Zhike Lu, Guoqing Li, Junjie Qin and the other users who tested the beta version of the program for identifying bugs and proposing all kinds of improvements. We thank Shengting Li for setting up the website. The project is supported by the National Natural Science Foundation of China (30725008), and grants from the Danish Natural Science Research Council (272-05-0344 and 272-07-0196).

Conflict of Interest: none declared.

REFERENCES

- Altschul,S.F. et al. (1997) Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Res.*, **25**, 3389–3402.
- Bentley,D.R. (2006) Whole-genome re-sequencing. *Curr. Opin. Genet. Dev.*, **16**, 545–552.
- Cox,A. (unpublished) ELAND: Efficient Local Alignment of Nucleotide Data.
- Kent,W.J. (2002) BLAT—the BLAST-like alignment tool. *Genome Res.*, **12**, 656–664.
- Li,H. (unpublished) Mapping and assembly with quality. <http://maq.sourceforge.net/>.
- Ning,Z. et al. (2001) SSAHA: a fast search method for large DNA databases. *Genome Res.*, **11**, 1725–1729.
- Shendure,J. et al. (2004) Advanced sequencing technologies: methods and goals. *Nat. Rev. Genet.*, **5**, 335–344.

Ultrafast and memory-efficient alignment of short DNA sequences to the human genome

Ben Langmead, Cole Trapnell, Mihai Pop and Steven L Salzberg

Address: Center for Bioinformatics and Computational Biology, Institute for Advanced Computer Studies, University of Maryland, College Park, MD 20742, USA.

Correspondence: Ben Langmead. Email: langmead@cs.umd.edu

Published: 4 March 2009

Genome Biology 2009, **10**:R25 (doi:10.1186/gb-2009-10-3-r25)

The electronic version of this article is the complete one and can be found online at <http://genomebiology.com/2009/10/3/R25>

Received: 21 October 2008

Revised: 19 December 2008

Accepted: 4 March 2009

© 2009 Langmead et al.; licensee BioMed Central Ltd.

This is an open access article distributed under the terms of the Creative Commons Attribution License (<http://creativecommons.org/licenses/by/2.0>), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Abstract

Bowtie is an ultrafast, memory-efficient alignment program for aligning short DNA sequence reads to large genomes. For the human genome, Burrows-Wheeler indexing allows Bowtie to align more than 25 million reads per CPU hour with a memory footprint of approximately 1.3 gigabytes. Bowtie extends previous Burrows-Wheeler techniques with a novel quality-aware backtracking algorithm that permits mismatches. Multiple processor cores can be used simultaneously to achieve even greater alignment speeds. Bowtie is open source <http://bowtie.cbcb.umd.edu>.

Rationale

Improvements in the efficiency of DNA sequencing have both broadened the applications for sequencing and dramatically increased the size of sequencing datasets. Technologies from Illumina (San Diego, CA, USA) and Applied Biosystems (Foster City, CA, USA) have been used to profile methylation patterns (MeDIP-Seq) [1], to map DNA-protein interactions (ChIP-Seq) [2], and to identify differentially expressed genes (RNA-Seq) [3] in the human genome and other species. The Illumina instrument was recently used to re-sequence three human genomes, one from a cancer patient and two from previously unsequenced ethnic groups [4-6]. Each of these studies required the alignment of large numbers of short DNA sequences ('short reads') onto the human genome. For example, two of the studies [4,5] used the short read alignment tool Maq [7] to align more than 130 billion bases (about 45× coverage) of short Illumina reads to a human reference genome in order to detect genetic variations. The third human re-sequencing study [6] used the SOAP program [8] to align more than 100 billion bases to the reference genome. In addition to these projects, the 1,000 Genomes project is in the process of using high-throughput sequencing instruments to

sequence a total of about six trillion base pairs of human DNA [9].

With existing methods, the computational cost of aligning many short reads to a mammalian genome is very large. For example, extrapolating from the results presented here in Tables 1 and 2, one can see that Maq would require more than 5 central processing unit (CPU)-months and SOAP more than 3 CPU-years to align the 140 billion bases from the study by Ley and coworkers [5]. Although using Maq or SOAP for this purpose has been shown to be feasible by using multiple CPUs, there is a clear need for new tools that consume less time and computational resources.

Maq and SOAP take the same basic algorithmic approach as other recent read mapping tools such as RMAP [10], ZOOM [11], and SHRiMP [12]. Each tool builds a hash table of short oligomers present in either the reads (SHRiMP, Maq, RMAP, and ZOOM) or the reference (SOAP). Some employ recent theoretical advances to align reads quickly without sacrificing sensitivity. For example, ZOOM uses 'spaced seeds' to significantly outperform RMAP, which is based on a simpler algo-

Table 1**Bowtie alignment performance versus SOAP and Maq**

	Platform	CPU time	Wall clock time	Reads mapped per hour (millions)	Peak virtual memory footprint (megabytes)	Bowtie speed-up	Reads aligned (%)
Bowtie -v 2	Server	15 m 7 s	15 m 41 s	33.8	1,149	-	67.4
SOAP		91 h 57 m 35 s	91 h 47 m 46 s	0.10	13,619	351×	67.3
Bowtie	PC	16 m 41 s	17 m 57 s	29.5	1,353	-	71.9
Maq		17 h 46 m 35 s	17 h 53 m 7 s	0.49	804	59.8×	74.7
Bowtie	Server	17 m 58 s	18 m 26 s	28.8	1,353	-	71.9
Maq		32 h 56 m 53 s	32 h 58 m 39 s	0.27	804	107×	74.7

The performance and sensitivity of Bowtie v0.9.6, SOAP v1.10, and Maq v0.6.6 when aligning 8.84 M reads from the 1,000 Genome project (National Center for Biotechnology Information Short Read Archive: SRR001115) trimmed to 35 base pairs. The 'soap.contig' version of the SOAP binary was used. SOAP could not be run on the PC because SOAP's memory footprint exceeds the PC's physical memory. For the SOAP comparison, Bowtie was invoked with '-v 2' to mimic SOAP's default matching policy (which allows up to two mismatches in the alignment and disregards quality values). For the Maq comparison Bowtie is run with its default policy, which mimics Maq's default policy of allowing up to two mismatches during the first 28 bases and enforcing an overall limit of 70 on the sum of the quality values at all mismatched positions. To make Bowtie's memory footprint more comparable to Maq's, Bowtie is invoked with the '-z' option in all experiments to ensure only the forward or mirror index is resident in memory at one time. CPU, central processing unit.

rithm developed by Baeza-Yates and Perleberg [13]. Spaced seeds have been shown to yield higher sensitivity than contiguous seeds of the same length [14,15]. SHRiMP employs a combination of spaced seeds and the Smith-Waterman [16] algorithm to align reads with high sensitivity at the expense of speed. Eland is a commercial alignment program available from Illumina that uses a hash-based algorithm to align reads.

Bowtie uses a different and novel indexing strategy to create an ultrafast, memory-efficient short read aligner geared toward mammalian re-sequencing. In our experiments using reads from the 1,000 Genomes project, Bowtie aligns 35-base pair (bp) reads at a rate of more than 25 million reads per CPU-hour, which is more than 35 times faster than Maq and 300 times faster than SOAP under the same conditions (see Tables 1 and 2). Bowtie employs a Burrows-Wheeler index based on the full-text minute-space (FM) index, which has a memory footprint of only about 1.3 gigabytes (GB) for the human genome. The small footprint allows Bowtie to run on

a typical desktop computer with 2 GB of RAM. The index is small enough to be distributed over the internet and to be stored on disk and re-used. Multiple processor cores can be used simultaneously to achieve even greater alignment speed. We have used Bowtie to align 14.3× coverage worth of human Illumina reads from the 1,000 Genomes project in about 14 hours on a single desktop computer with four processor cores.

Bowtie makes a number of compromises to achieve this speed, but these trade-offs are reasonable within the context of mammalian re-sequencing projects. If one or more exact matches exist for a read, then Bowtie is guaranteed to report one, but if the best match is an inexact one then Bowtie is not guaranteed in all cases to find the highest quality alignment. With its highest performance settings, Bowtie may fail to align a small number of reads with valid alignments, if those reads have multiple mismatches. If the stronger guarantees are desired, Bowtie supports options that increase accuracy at the cost of some performance. For instance, the '--best' option will guarantee that all alignments reported are best in terms

Table 2**Bowtie alignment performance versus Maq with filtered read set**

	Platform	CPU time	Wall clock time	Reads mapped per hour (millions)	Peak virtual memory footprint (megabytes)	Bowtie speed up	Reads aligned (%)
Bowtie	PC	16 m 39 s	17 m 47 s	29.8	1,353	-	74.9
Maq		11 h 15 m 58 s	11 h 22 m 2 s	0.78	804	38.4×	78.0
Bowtie	Server	18 m 20 s	18 m 46 s	28.3	1,352	-	74.9
Maq		18 h 49 m 7 s	18 h 50 m 16 s	0.47	804	60.2×	78.0

Performance and sensitivity of Bowtie v0.9.6 and Maq v0.6.6 when the read set is filtered using Maq's 'catfilter' command to eliminate poly-A artifacts. The filter eliminates 438,145 out of 8,839,010 reads. Other experimental parameters are identical to those of the experiments in Table 1. CPU, central processing unit.

of minimizing mismatches in the seed portion of the read, although this option incurs additional computational cost.

With its default options, Bowtie's sensitivity measured in terms of reads aligned is equal to SOAP's and somewhat less than Maq's. Command line options allow the user to increase sensitivity at the cost of greater running time, and to enable Bowtie to report multiple hits for a read. Bowtie can align reads as short as four bases and as long as 1,024 bases. The input to a single run of Bowtie may comprise a mixture of reads with different lengths.

Bowtie description and results

Bowtie indexes the reference genome using a scheme based on the Burrows-Wheeler transform (BWT) [17] and the FM index [18,19]. A Bowtie index for the human genome fits in 2.2 GB on disk and has a memory footprint of as little as 1.3 GB at alignment time, allowing it to be queried on a workstation with under 2 GB of RAM.

The common method for searching in an FM index is the exact-matching algorithm of Ferragina and Manzini [18]. Bowtie does not simply adopt this algorithm because exact matching does not allow for sequencing errors or genetic variations. We introduce two novel extensions that make the technique applicable to short read alignment: a quality-aware backtracking algorithm that allows mismatches and favors high-quality alignments; and 'double indexing', a strategy to

avoid excessive backtracking. The Bowtie aligner follows a policy similar to Maq's, in that it allows a small number of mismatches within the high-quality end of each read, and it places an upper limit on the sum of the quality values at mismatched alignment positions.

Burrows-Wheeler indexing

The BWT is a reversible permutation of the characters in a text. Although originally developed within the context of data compression, BWT-based indexing allows large texts to be searched efficiently in a small memory footprint. It has been applied to bioinformatics applications, including oligomer counting [20], whole-genome alignment [21], tiling microarray probe design [22], and Smith-Waterman alignment to a human-sized reference [23].

The Burrows-Wheeler transformation of a text T , $BWT(T)$, is constructed as follows. The character $\$$ is appended to T , where $\$$ is not in T and is lexicographically less than all characters in T . The Burrows-Wheeler matrix of T is constructed as the matrix whose rows comprise all cyclic rotations of $T\$$. The rows are then sorted lexicographically. $BWT(T)$ is the sequence of characters in the rightmost column of the Burrows-Wheeler matrix (Figure 1a). $BWT(T)$ has the same length as the original text T .

This matrix has a property called 'last first (LF) mapping'. The i^{th} occurrence of character X in the last column corresponds to the same text character as the i^{th} occurrence of X in the first

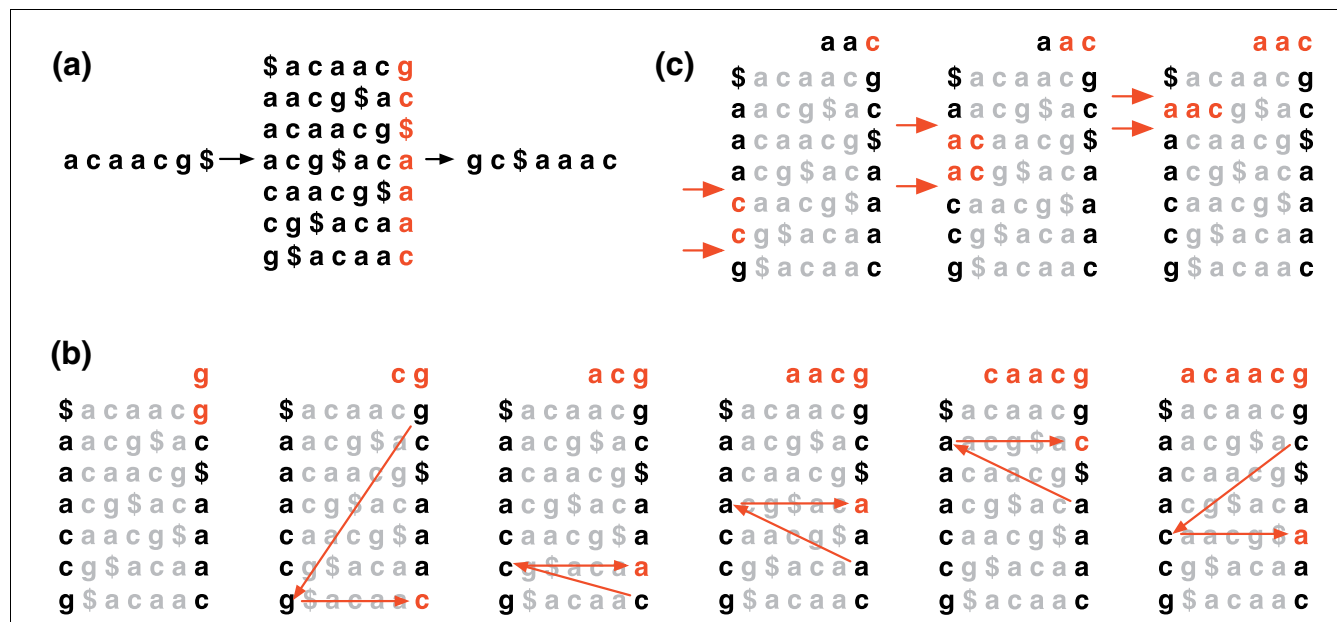


Figure 1

Burrows-Wheeler transform. (a) The Burrows-Wheeler matrix and transformation for 'acaacg'. (b) Steps taken by EXACTMATCH to identify the range of rows, and thus the set of reference suffixes, prefixed by 'aac'. (c) UNPERMUTE repeatedly applies the last first (LF) mapping to recover the original text (in red on the top line) from the Burrows-Wheeler transform (in black in the rightmost column).

column. This property underlies algorithms that use the BWT index to navigate or search the text. Figure 1b illustrates UNPERMUTE, an algorithm that applies the LF mapping repeatedly to re-create T from BWT(T).

The LF mapping is also used in exact matching. Because the matrix is sorted lexicographically, rows beginning with a given sequence appear consecutively. In a series of steps, the EXACTMATCH algorithm (Figure 1c) calculates the range of matrix rows beginning with successively longer suffixes of the query. At each step, the size of the range either shrinks or remains the same. When the algorithm completes, rows beginning with S_0 (the entire query) correspond to exact occurrences of the query in the text. If the range is empty, the text does not contain the query. UNPERMUTE is attributable to Burrows and Wheeler [17] and EXACTMATCH to Ferragina and Manzini [18]. See Additional data file 1 (Supplemental Discussion 1) for details.

Searching for inexact alignments

EXACTMATCH is insufficient for short read alignment because alignments may contain mismatches, which may be due to sequencing errors, genuine differences between reference and query organisms, or both. We introduce an alignment algorithm that conducts a backtracking search to quickly find alignments that satisfy a specified alignment policy. Each character in a read has a numeric quality value, with lower values indicating a higher likelihood of a sequencing error. Our alignment policy allows a limited number of mismatches and prefers alignments where the sum of the quality values at all mismatched positions is low.

The search proceeds similarly to EXACTMATCH, calculating matrix ranges for successively longer query suffixes. If the range becomes empty (a suffix does not occur in the text), then the algorithm may select an already-matched query position and substitute a different base there, introducing a mismatch into the alignment. The EXACTMATCH search resumes from just after the substituted position. The algorithm selects only those substitutions that are consistent with the alignment policy and which yield a modified suffix that occurs at least once in the text. If there are multiple candidate substitution positions, then the algorithm greedily selects a position with a minimal quality value.

Backtracking scenarios play out within the context of a stack structure that grows when a new substitution is introduced and shrinks when the aligner rejects all candidate alignments for the substitutions currently on the stack. See Figure 2 for an illustration of how the search might proceed.

In short, Bowtie conducts a quality-aware, greedy, randomized, depth-first search through the space of possible alignments. If a valid alignment exists, then Bowtie will find it (subject to the backtrack ceiling discussed in the following section). Because the search is greedy, the first valid align-

ment encountered by Bowtie will not necessarily be the 'best' in terms of number of mismatches or in terms of quality. The user may instruct Bowtie to continue searching until it can prove that any alignment it reports is 'best' in terms of number of mismatches (using the option `--best`). In our experience, this mode is two to three times slower than the default mode. We expect that the faster default mode will be preferred for large re-sequencing projects.

The user may also opt for Bowtie to report all alignments up to a specified number (option `-k`) or all alignments with no limit on the number (option `-a`) for a given read. If in the course of its search Bowtie finds N possible alignments for a given set of substitutions, but the user has requested only K alignments where $K < N$, Bowtie will report K of the N alignments selected at random. Note that these modes can be much slower than the default. In our experience, for example, `-k 1` is more than twice as fast as `-k 2`.

Excessive backtracking

The aligner as described so far can, in some cases, encounter sequences that cause excessive backtracking. This occurs when the aligner spends most of its effort fruitlessly backtracking to positions close to the 3' end of the query. Bowtie mitigates excessive backtracking with the novel technique of 'double indexing'. Two indices of the genome are created: one containing the BWT of the genome, called the 'forward' index, and a second containing the BWT of the genome with its character sequence reversed (not reverse complemented) called the 'mirror' index. To see how this helps, consider a matching policy that allows one mismatch in the alignment. A valid alignment with one mismatch falls into one of two cases according to which half of the read contains the mismatch. Bowtie proceeds in two phases corresponding to those two cases. Phase 1 loads the forward index into memory and invokes the aligner with the constraint that it may not substitute at positions in the query's right half. Phase 2 uses the mirror index and invokes the aligner on the reversed query, with the constraint that the aligner may not substitute at positions in the reversed query's right half (the original query's left half). The constraints on backtracking into the right half prevent excessive backtracking, whereas the use of two phases and two indices maintains full sensitivity.

Unfortunately, it is not possible to avoid excessive backtracking fully when alignments are permitted to have two or more mismatches. In our experiments, we have observed that excessive backtracking is significant only when a read has many low-quality positions and does not align or aligns poorly to the reference. These cases can trigger in excess of 200 backtracks per read because there are many legal combinations of low-quality positions to be explored before all possibilities are exhausted. We mitigate this cost by enforcing a limit on the number of backtracks allowed before a search is terminated (default: 125). The limit prevents some legitimate,

low-quality alignments from being reported, but we expect that this is a desirable trade-off for most applications.

Phased Maq-like search

Bowtie allows the user to select the number of mismatches permitted (default: two) in the high-quality end of a read (default: the first 28 bases) as well as the maximum acceptable quality distance of the overall alignment (default: 70). Quality values are assumed to follow the definition in PHRED [24], where p is the probability of error and $Q = -10 \log p$.

Both the read and its reverse complement are candidates for alignment to the reference. For clarity, this discussion considers only the forward orientation. See Additional data file 1 (Supplementary Discussion 2) for an explanation of how the reverse complement is incorporated.

The first 28 bases on the high-quality end of the read are termed the 'seed'. The seed consists of two halves: the 14 bp on the high-quality end (usually the 5' end) and the 14 bp on the low-quality end, termed the 'hi-half' and the 'lo-half', respectively. Assuming the default policy (two mismatches permitted in the seed), a reportable alignment will fall into one of four cases: no mismatches in seed (case 1); no mismatches in hi-half, one or two mismatches in lo-half (case 2); no mismatches in lo-half, one or two mismatches in hi-half (case 3); and one mismatch in hi-half, one mismatch in lo-half (case 4).

All cases allow any number of mismatches in the nonseed part of the read and all cases are also subject to the quality distance constraint.

The Bowtie algorithm consists of three phases that alternate between using the forward and mirror indices, as illustrated in Figure 3. Phase 1 uses the mirror index and invokes the aligner to find alignments for cases 1 and 2. Phases 2 and 3 cooperate to find alignments for case 3: Phase 2 finds partial alignments with mismatches only in the hi-half and phase 3 attempts to extend those partial alignments into full alignments. Finally, phase 3 invokes the aligner to find alignments for case 4.

Performance results

We evaluated the performance of Bowtie using reads from the 1,000 Genomes project pilot (National Center for Biotechnology Information [NCBI] Short Read Archive:SRRO01115). A total of 8.84 million reads, about one lane of data from an Illumina instrument, were trimmed to 35 bp and aligned to the human reference genome [NCBI build 36.3]. Unless specified otherwise, read data are not filtered or modified (besides trimming) from how they appear in the archive. This leads to about 70% to 75% of reads aligning somewhere to the genome. In our experience, this is typical for raw data from the archive. More aggressive filtering leads to higher alignment rates and faster alignment.

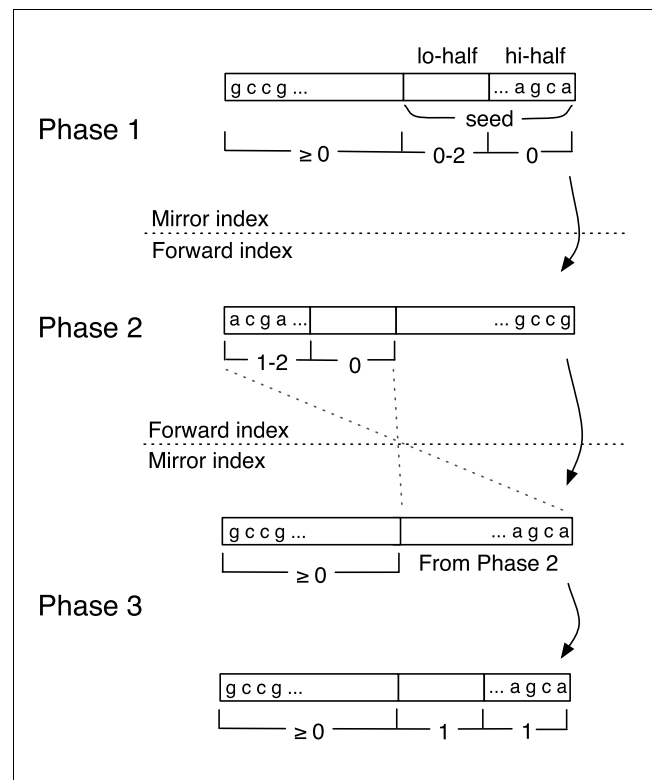


Figure 3

The three phases of the Bowtie algorithm for the Maq-like policy. A three-phase approach finds alignments for two-mismatch cases 1 to 4 while minimizing backtracking. Phase 1 uses the mirror index and invokes the aligner to find alignments for cases 1 and 2. Phases 2 and 3 cooperate to find alignments for case 3: Phase 2 finds partial alignments with mismatches only in the hi-half and phase 3 attempts to extend those partial alignments into full alignments. Finally, phase 3 invokes the aligner to find alignments for case 4.

All runs were performed on a single CPU. Bowtie speedups were calculated as a ratio of wall-clock alignment times. Both wall-clock and CPU times are given to demonstrate that input/output load and CPU contention are not significant factors.

The time required to build the Bowtie index was not included in the Bowtie running times. Unlike competing tools, Bowtie can reuse a pre-computed index for the reference genome across many alignment runs. We anticipate most users will simply download such indices from a public repository. The Bowtie site [25] provides indices for current builds of the human, chimp, mouse, dog, rat, and *Arabidopsis thaliana* genomes, as well as many others.

Results were obtained on two hardware platforms: a desktop workstation with 2.4 GHz Intel Core 2 processor and 2 GB of RAM; and a large-memory server with a four-core 2.4 GHz AMD Opteron processor and 32 GB of RAM. These are denoted 'PC' and 'server', respectively. Both PC and server run Red Hat Enterprise Linux AS release 4.

Comparison to SOAP and Maq

Maq is a popular aligner [1,4,5,26,27] that is among the fastest competing open source tools for aligning millions of Illumina reads to the human genome. SOAP is another open source tool that has been reported and used in short-read projects [6,28].

Table 1 presents the performance and sensitivity of Bowtie vo.9.6, SOAP v1.10, and Maq vo.6.6. SOAP could not be run on the PC because SOAP's memory footprint exceeds the PC's physical memory. The 'soap.contig' version of the SOAP binary was used. For comparison with SOAP, Bowtie was invoked with '-v 2' to mimic SOAP's default matching policy (which allows up to two mismatches in the alignment and disregards quality values), and with '--maxns 5' to simulate SOAP's default policy of filtering out reads with five or more no-confidence bases. For the Maq comparison Bowtie is run with its default policy, which mimics Maq's default policy of allowing up to two mismatches in the first 28 bases and enforcing an overall limit of 70 on the sum of the quality values at all mismatched read positions. To make Bowtie's memory footprint more comparable to Maq's, Bowtie is invoked with the '-z' option in all experiments to ensure that only the forward or mirror index is resident in memory at one time.

The number of reads aligned indicates that SOAP (67.3%) and Bowtie -v 2 (67.4%) have comparable sensitivity. Of the reads aligned by either SOAP or Bowtie, 99.7% were aligned by both, 0.2% were aligned by Bowtie but not SOAP, and 0.1% were aligned by SOAP but not Bowtie. Maq (74.7%) and Bowtie (71.9%) also have roughly comparable sensitivity, although Bowtie lags by 2.8%. Of the reads aligned by either Maq or Bowtie, 96.0% were aligned by both, 0.1% were aligned by Bowtie but not Maq, and 3.9% were aligned by Maq but not Bowtie. Of the reads mapped by Maq but not Bowtie, almost all are due to a flexibility in Maq's alignment algorithm that allows some alignments to have three mismatches in the seed. The remainder of the reads mapped by Maq but not Bowtie are due to Bowtie's backtracking ceiling.

Maq's documentation mentions that reads containing 'poly-A artifacts' can impair Maq's performance. Table 2 presents performance and sensitivity of Bowtie and Maq when the read set is filtered using Maq's 'catfilter' command to eliminate poly-A artifacts. The filter eliminates 438,145 out of 8,839,010 reads. Other experimental parameters are identical to those of the experiments in Table 1, and the same observations about the relative sensitivity of Bowtie and Maq apply here.

Read length and performance

As sequencing technology improves, read lengths are growing beyond the 30-bp to 50-bp commonly seen in public databases today. Bowtie, Maq, and SOAP support reads of lengths up to 1,024, 63, and 60 bp, respectively, and Maq versions 0.7.0 and later support read lengths up to 127 bp. Table 3

shows performance results when the three tools are each used to align three sets of 2 M untrimmed reads, a 36-bp set, a 50-bp set and a 76-bp set, to the human genome on the server platform. Each set of 2 M is randomly sampled from a larger set (NCBI Short Read Archive: SRR003084 for 36-bp, SRR003092 for 50-bp, SRR003196 for 76-bp). Reads were sampled such that the three sets of 2 M have uniform per-base error rate, as calculated from per-base Phred qualities. All reads pass through Maq's 'catfilter'.

Bowtie is run both in its Maq-like default mode and in its SOAP-like '-v 2' mode. Bowtie is also given the '-z' option to ensure that only the forward or mirror index is resident in memory at one time. Maq vo.7.1 was used instead of Maq vo.6.6 for the 76-bp set because vo.6.6 cannot align reads longer than 63 bp. SOAP was not run on the 76-bp set because it does not support reads longer than 60 bp.

The results show that Maq's algorithm scales better overall to longer read lengths than Bowtie or SOAP. However, Bowtie in SOAP-like '-v 2' mode also scales very well. Bowtie in its default Maq-like mode scales well from 36-bp to 50-bp reads but is substantially slower for 76-bp reads, although it is still more than an order of magnitude faster than Maq.

Parallel performance

Alignment can be parallelized by distributing reads across concurrent search threads. Bowtie allows the user to specify a desired number of threads (option -p); Bowtie then launches the specified number of threads using the pthreads library. Bowtie threads synchronize with each other when fetching reads, outputting results, switching between indices, and performing various forms of global bookkeeping, such as marking a read as 'done'. Otherwise, threads are free to operate in parallel, substantially speeding up alignment on computers with multiple processor cores. The memory image of the index is shared by all threads, and so the footprint does not increase substantially when multiple threads are used. Table 4 shows performance results for running Bowtie vo.9.6 on the four-core server with one, two, and four threads.

Index building

Bowtie uses a flexible indexing algorithm [29] that can be configured to trade off between memory usage and running time. Table 5 illustrates this trade-off when indexing the entire human reference genome (NCBI build 36.3, contigs). Runs were performed on the server platform. The indexer was run four times with different upper limits on memory usage.

The reported times compare favorably with alignment times of competing tools that perform indexing during alignment. Less than 5 hours is required for Bowtie to both build and query a whole-human index with 8.84 million reads from the 1,000 Genome project (NCBI Short Read Archive:SRR001115) on a server, more than sixfold faster than the equivalent Maq run. The bottom-most row illus-

Table 3**Varying read length using Bowtie, Maq and SOAP**

Length	Program	CPU time	Wall clock time	Peak virtual memory footprint (megabytes)	Bowtie speed-up	Reads aligned (%)
36 bp	Bowtie	6 m 15 s	6 m 21 s	1,305	-	62.2
	Maq	3 h 52 m 26 s	3 h 52 m 54 s	804	36.7×	65.0
	Bowtie -v 2	4 m 55 s	5 m 00 s	1,138	-	55.0
	SOAP	16 h 44 m 3 s	18 h 1 m 38 s	13,619	216×	55.1
50 bp	Bowtie	7 m 11 s	7 m 20 s	1,310	-	67.5
	Maq	2 h 39 m 56 s	2 h 40 m 9 s	804	21.8×	67.9
	Bowtie -v 2	5 m 32 s	5 m 46 s	1,138	-	56.2
	SOAP	48 h 42 m 4 s	66 h 26 m 53 s	13,619	691×	56.2
76 bp	Bowtie	18 m 58 s	19 m 6 s	1,323	-	44.5
	Maq 0.7.1	4 h 45 m 7 s	4 h 45 m 17 s	1,155	14.9×	44.9
	Bowtie -v 2	7 m 35 s	7 m 40 s	1,138	-	31.7

The performance of Bowtie v0.9.6, SOAP v1.10, and Maq versions v0.6.6 and v0.7.1 on the server platform when aligning 2 M untrimmed reads from the 1,000 Genome project (National Center for Biotechnology Information Short Read Archive: SRR003084 for 36 base pairs [bp], SRR003092 for 50 bp, and SRR003196 for 76 bp). For each read length, the 2 M reads were randomly sampled from the FASTQ file downloaded from the Archive such that the average per-base error rate as measured by quality values was uniform across the three sets. All reads pass through Maq's "catfilter". Maq v0.7.1 was used for the 76-bp reads because v0.6.6 does not support reads longer than 63 bp. SOAP is excluded from the 76-bp experiment because it does not support reads longer than 60 bp. Other experimental parameters are identical to those of the experiments in Table 1. CPU, central processing unit.

trates that the Bowtie indexer, with appropriate arguments, is memory-efficient enough to run on a typical workstation with 2 GB of RAM. Additional data file 1 (Supplementary discussions 3 and 4) explains the algorithm and the contents of the resulting index.

Software

Bowtie is written in C++ and uses the SeqAn library [30]. The converter to the Maq mapping format uses code from Maq.

Discussion

Bowtie exhibits a large performance advantage over both Maq and SOAP when mapping reads to the human genome. Bowtie's sensitivity in terms of reads aligned is comparable to that of SOAP and slightly less than Maq's, although the user may use command-line options to trade slower running time for greater sensitivity. Unlike SOAP, Bowtie's 1.3 GB memory footprint allows it to run on a typical PC with 2 GB of RAM. Bowtie aligns Illumina reads to the human genome at a rate of over 25 million reads per hour. Multiple processor cores can run parallel Bowtie threads to achieve even greater align-

Table 4**Bowtie parallel alignment performance**

	CPU time	Wall clock time	Reads mapped per hour (millions)	Peak virtual memory footprint (megabytes)	Speedup
Bowtie, one thread	18 m 19 s	18 m 46 s	28.3	1,353	-
Bowtie, two threads	20 m 34 s	10 m 35 s	50.1	1,363	1.77×
Bowtie, four threads	23 m 9 s	6 m 1 s	88.1	1,384	3.12×

Performance results for running Bowtie v0.9.6 on the four-core server with one, two, and four threads. Other experimental parameters are identical to those of the experiments in Table 1. CPU, central processing unit.

Table 5**Bowtie index building performance**

Physical memory target (GB)	Actual peak memory footprint (GB)	Wall clock time
16	14.4	4 h 36 m
8	5.84	5 h 5 m
4	3.39	7 h 40 m
2	1.39	21 h 30 m

Performance results and memory footprints of running the Bowtie v0.9.6 indexer on the whole human genome (National Center for Biotechnology Information build 36.3, contigs). Runs were performed on the server platform. The indexer was run four times with different upper limits on memory usage. See Additional data file 1 (Supplementary Discussion 3 and Supplementary Table 1) for details.

ment speed; experiments show a speed up of 3.12 for four threads on a typical Opteron server.

Unlike many other short-read aligners, Bowtie creates a permanent index of the reference that may be re-used across alignment runs. Building the index is fast - Bowtie outperforms competing tools when aligning lanes of Illumina reads even with index construction time included. At 2.2 GB for the human genome, the on-disk size of a Bowtie index is small enough to distribute over the internet. The Bowtie website hosts pre-built indices for the human genome and several other model organisms including chimp, dog, rat, mouse, and chicken.

Bowtie's speed and small memory footprint are due chiefly to its use of the Burrows-Wheeler index in combination with the novel, quality-aware, backtracking algorithm introduced here. Double indexing is used to avoid the performance penalty of excessive backtracking.

Bowtie supports standard FASTQ and FASTA input formats, and comes with a conversion program that allows Bowtie output to be used with Maq's consensus generator and single nucleotide polymorphism caller.

Bowtie does not yet support paired-end alignment or alignments with insertions or deletions, although both improvements are planned for the future. Paired-end alignment is not difficult to implement in Bowtie's framework, and we expect that Bowtie's performance advantage will be comparable to, though perhaps somewhat less than, that of unpaired alignment mode. Support for insertions and deletions is also a conceptually straightforward addition.

Bowtie is free, open source software available from the Bowtie website [25].

Abbreviations

bp: base pair; BWT: Burrows-Wheeler transform; CPU: central processing unit; FM: full-text minute-space; GB: giga-

bytes; LF: last first; NCBI: National Center for Biotechnology Information.

Authors' contributions

BL developed the algorithms, collected results, and wrote most of the software. CT wrote some of the software. CT and MP contributed to discussions on algorithms. BL, CT, MP, and SLS wrote the manuscript.

Additional data files

The following additional data are included with the online version of this article: a document containing supplementary discussions, tables, and figures pertaining to algorithms for navigating the Burrows-Wheeler transform, the full four-phase version of the alignment algorithm that incorporates the reverse-complement, index construction, and components of the index (Additional data file 1).

Acknowledgements

The authors would like to thank Arthur Delcher for his thorough review of the manuscript, and Michael Schatz for helpful discussion of algorithms. This research was supported in part by NIH grants R01-LM006845 and R01-GM083873 to SLS.

References

- Down TA, Rakyen VK, Turner DJ, Flicek P, Li H, Kulesha E, Graf S, Johnson N, Herrero J, Tomazou EM, Thorne NP, Backdahl L, Herberth M, Howe KL, Jackson DK, Miretti MM, Marioni JC, Birney E, Hubbard TJ, Durbin R, Tavare S, Beck S: **A Bayesian deconvolution strategy for immunoprecipitation-based DNA methylation analysis.** *Nat Biotechnol* 2008, **26**:779-785.
- Johnson DS, Mortazavi A, Myers RM, Wold B: **Genome-wide mapping of in vivo protein-DNA interactions.** *Science* 2007, **316**:1497-1502.
- Marioni JC, Mason CE, Mane SM, Stephens M, Gilad Y: **RNA-seq: an assessment of technical reproducibility and comparison with gene expression arrays.** *Genome Res* 2008, **18**:1509-1517.
- Bentley DR, Balasubramanian S, Swerdlow HP, Smith GP, Milton J, Brown CG, Hall KP, Evers DJ, Barnes CL, Bignell HR, Boutell JM, Bryant J, Carter RJ, Keira Cheetham R, Cox AJ, Ellis DJ, Flatbush MR, Gormley NA, Humphray SJ, Irving LJ, Karabashvili MS, Kirk SM, Li H, Liu X, Maisinger KS, Murray LJ, Obradovic B, Ost T, Parkinson ML, Pratt MR, et al.: **Accurate whole human genome sequencing using reversible terminator chemistry.** *Nature* 2008, **456**:53-59.

5. Ley TJ, Mardis ER, Ding L, Fulton B, McLellan MD, Chen K, Dooling D, Dunford-Shore BH, McGrath S, Hickenbotham M, Cook L, Abbott R, Larson DE, Koboldt DC, Pohl C, Smith S, Hawkins A, Abbott S, Locke D, Hillier LW, Miner T, Fulton L, Magrini V, Wylie T, Glasscock J, Conyers J, Sander N, Shi X, Osborne JR, Minx P, et al.: **DNA sequencing of a cytogenetically normal acute myeloid leukaemia genome.** *Nature* 2008, **456**:66-72.
6. Wang J, Wang W, Li R, Li Y, Tian G, Goodman L, Fan W, Zhang J, Li J, Zhang J, Guo Y, Feng B, Li H, Lu Y, Fang X, Liang H, Du Z, Li D, Zhao Y, Hu Y, Yang Z, Zheng H, Hellmann I, Inouye M, Pool J, Yi X, Zhao J, Duan J, Zhou Y, Qin J, et al.: **The diploid genome sequence of an Asian individual.** *Nature* 2008, **456**:60-65.
7. Li H, Ruan J, Durbin R: **Mapping short DNA sequencing reads and calling variants using mapping quality scores.** *Genome Res* 2008, **18**:1851-1858.
8. Li R, Li Y, Kristiansen K, Wang J: **SOAP: short oligonucleotide alignment program.** *Bioinformatics* 2008, **24**:713-714.
9. Kaiser J: **DNA sequencing. A plan to capture human diversity in 1000 genomes.** *Science* 2008, **319**:395.
10. Smith AD, Xuan Z, Zhang MQ: **Using quality scores and longer reads improves accuracy of Solexa read mapping.** *BMC Bioinformatics* 2008, **9**:128.
11. Lin H, Zhang Z, Zhang MQ, Ma B, Li M: **ZOOM! Zillions Of Oligos Mapped.** *Bioinformatics* 2008, **24**:2431-2437.
12. **SHRIMP - SHort Read Mapping Package** [<http://comp.bio.cs.toronto.edu/shrimp/>]
13. Baeza-Yates RA, Perleberg CH: **Fast and practical approximate string matching.** *Inf Process Lett* 1996, **59**:21-27.
14. Burkhardt S, Kärkkäinen J: **Better Filtering with Gapped q-Grams.** *Fundam Inf* 2003, **56**:51-70.
15. Ma B, Tromp J, Li M: **PatternHunter: faster and more sensitive homology search.** *Bioinformatics* 2002, **18**:440-445.
16. Smith TF, Waterman MS: **Identification of common molecular subsequences.** *J Mol Biol* 1981, **147**:195-197.
17. Burrows M, Wheeler DJ: *A Block Sorting Lossless Data Compression Algorithm. Technical Report 124* Palo Alto, CA: Digital Equipment Corporation; 1994.
18. Ferragina P, Manzini G: **Opportunistic data structures with applications.** [<http://web.unipmn.it/~manzini/papers/focs00draft.pdf>].
19. Ferragina P, Manzini G: **An experimental study of an opportunistic index.** In *Proceedings of the Twelfth Annual ACM-SIAM Symposium on Discrete algorithms* Washington, DC: Society for Industrial and Applied Mathematics; 2001:269-278.
20. Healy J, Thomas EE, Schwartz JT, Wigler M: **Annotating large genomes with exact word matches.** *Genome Res* 2003, **13**:2306-2315.
21. Lippert RA: **Space-efficient whole genome comparisons with Burrows-Wheeler transforms.** *J Comput Biol* 2005, **12**:407-415.
22. Graf S, Nielsen FG, Kurtz S, Huynen MA, Birney E, Stunnenberg H, Flícek P: **Optimized design and assessment of whole genome tiling arrays.** *Bioinformatics* 2007, **23**:i195-i204.
23. Lam TW, Sung WK, Tam SL, Wong CK, Yiu SM: **Compressed indexing and local alignment of DNA.** *Bioinformatics* 2008, **24**:791-797.
24. Ewing B, Green P: **Base-calling of automated sequencer traces using phred. II. Error probabilities.** *Genome Res* 1998, **8**:186-194.
25. **Bowtie: An ultrafast memory-efficient short read aligner** [<http://bowtie.cbcb.umd.edu/>]
26. Campbell PJ, Stephens PJ, Pleasance ED, O'Meara S, Li H, Santarius T, Stebbings LA, Leroy C, Edkins S, Hardy C, Teague JW, Menzies A, Goodhead I, Turner DJ, Clee CM, Quail MA, Cox A, Brown C, Durbin R, Hurler ME, Edwards PA, Bignell GR, Stratton MR, Futreal PA: **Identification of somatically acquired rearrangements in cancer using genome-wide massively parallel paired-end sequencing.** *Nat Genet* 2008, **40**:722-729.
27. Holt KE, Parkhill J, Mazzoni CJ, Roumagnac P, Weill FX, Goodhead I, Rance R, Baker S, Maskell DJ, Wain J, Dolecek C, Achtman M, Dougan G: **High-throughput sequencing provides insights into genome variation and evolution in *Salmonella typhi*.** *Nat Genet* 2008, **40**:987-993.
28. Nagalakshmi U, Wang Z, Waern K, Shou C, Raha D, Gerstein M, Snyder M: **The transcriptional landscape of the yeast genome defined by RNA sequencing.** *Science* 2008, **320**:1344-1349.
29. Kärkkäinen J: **Fast BWT in small space by blockwise suffix sorting.** *Theor Comput Sci* 2007, **387**:249-257.
30. Döring A, Weese D, Rausch T, Reinert K: **SeqAn an efficient, generic C++ library for sequence analysis.** *BMC Bioinformatics* 2008, **9**:11.

Sequence analysis

Fast and accurate short read alignment with Burrows–Wheeler transform

Heng Li and Richard Durbin*

Wellcome Trust Sanger Institute, Wellcome Trust Genome Campus, Cambridge, CB10 1SA, UK

Received on February 20, 2009; revised on May 6, 2009; accepted on May 12, 2009

Advance Access publication May 18, 2009

Associate Editor: John Quackenbush

ABSTRACT

Motivation: The enormous amount of short reads generated by the new DNA sequencing technologies call for the development of fast and accurate read alignment programs. A first generation of hash table-based methods has been developed, including MAQ, which is accurate, feature rich and fast enough to align short reads from a single individual. However, MAQ does not support gapped alignment for single-end reads, which makes it unsuitable for alignment of longer reads where indels may occur frequently. The speed of MAQ is also a concern when the alignment is scaled up to the resequencing of hundreds of individuals.

Results: We implemented Burrows–Wheeler Alignment tool (BWA), a new read alignment package that is based on backward search with Burrows–Wheeler Transform (BWT), to efficiently align short sequencing reads against a large reference sequence such as the human genome, allowing mismatches and gaps. BWA supports both base space reads, e.g. from Illumina sequencing machines, and color space reads from AB SOLiD machines. Evaluations on both simulated and real data suggest that BWA is ~10–20× faster than MAQ, while achieving similar accuracy. In addition, BWA outputs alignment in the new standard SAM (Sequence Alignment/Map) format. Variant calling and other downstream analyses after the alignment can be achieved with the open source SAMtools software package.

Availability: <http://maq.sourceforge.net>

Contact: rd@sanger.ac.uk

1 INTRODUCTION

The Illumina/Solexa sequencing technology typically produces 50–200 million 32–100 bp reads on a single run of the machine. Mapping this large volume of short reads to a genome as large as human poses a great challenge to the existing sequence alignment programs. To meet the requirement of efficient and accurate short read mapping, many new alignment programs have been developed. Some of these, such as Eland (Cox, 2007, unpublished material), RMAP (Smith *et al.*, 2008), MAQ (Li *et al.*, 2008a), ZOOM (Lin *et al.*, 2008), SeqMap (Jiang and Wong, 2008), CloudBurst (Schatz, 2009) and SHRiMP (<http://compbio.cs.toronto.edu/shrimp>), work by hashing the read sequences and scan through the reference sequence. Programs in this category usually have flexible memory footprint, but may have the overhead

of scanning the whole genome when few reads are aligned. The second category of software, including SOAPv1 (Li *et al.*, 2008b), PASS (Campagna *et al.*, 2009), MOM (Eaves and Gao, 2009), ProbeMatch (Jung Kim *et al.*, 2009), NovoAlign (<http://www.novocraft.com>), ReSEQ (<http://code.google.com/p/re-seq>), Mosaik (<http://bioinformatics.bc.edu/marthlab/Mosaik>) and BFAST (<http://genome.ucla.edu/bfast>), hash the genome. These programs can be easily parallelized with multi-threading, but they usually require large memory to build an index for the human genome. In addition, the iterative strategy frequently introduced by these software may make their speed sensitive to the sequencing error rate. The third category includes slider (Malhis *et al.*, 2009) which does alignment by merge-sorting the reference subsequences and read sequences.

Recently, the theory on string matching using Burrows–Wheeler Transform (BWT) (Burrows and Wheeler, 1994) has drawn the attention of several groups, which has led to the development of SOAPv2 (<http://soap.genomics.org.cn/>), Bowtie (Langmead *et al.*, 2009) and BWA, our new aligner described in this article. Essentially, using backward search (Ferragina and Manzini, 2000; Lippert, 2005) with BWT, we are able to effectively mimic the top-down traversal on the prefix trie of the genome with relatively small memory footprint (Lam *et al.*, 2008) and to count the number of exact hits of a string of length m in $O(m)$ time independent of the size of the genome. For inexact search, BWA samples from the implicit prefix trie the distinct substrings that are less than k edit distance away from the query read. Because exact repeats are collapsed on one path on the prefix trie, we do not need to align the reads against each copy of the repeat. This is the main reason why BWT-based algorithms are efficient.

In this article, we will give a sufficient introduction to the background of BWT and backward search for exact matching, and present the algorithm for inexact matching which is implemented in BWA. We evaluate the performance of BWA on simulated data by comparing the BWA alignment with the true alignment from the simulation, as well as on real paired-end data by checking the fraction of reads mapped in consistent pairs and by counting misaligned reads mapped against a hybrid genome.

2 METHODS

2.1 Prefix trie and string matching

The prefix trie for string X is a tree where each edge is labeled with a symbol and the string concatenation of the edge symbols on the path from a leaf to

*To whom correspondence should be addressed.

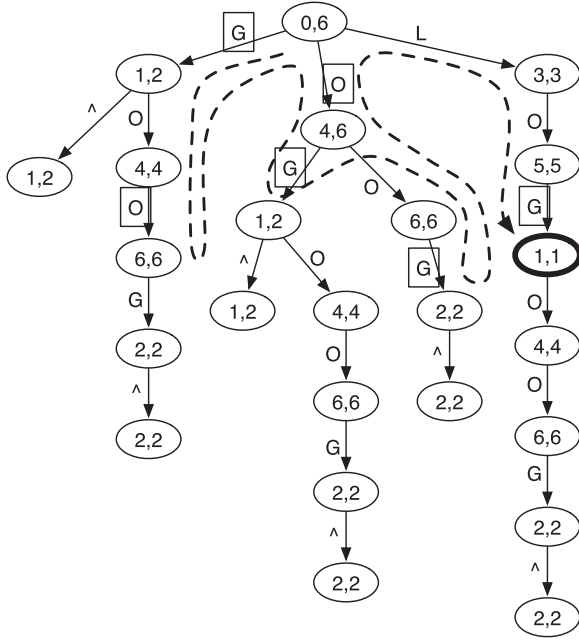


Fig. 1. Prefix trie of string 'GOOGOL'. Symbol $\wedge$ marks the start of the string. The two numbers in a node give the SA interval of the string represented by the node (see Section 2.3). The dashed line shows the route of the brute-force search for a query string 'LOL', allowing at most one mismatch. Edge labels in squares mark the mismatches to the query in searching. The only hit is the bold node [1, 1] which represents string 'GOL'.

the root gives a unique prefix of X . On the prefix trie, the string concatenation of the edge symbols from a node to the root gives a unique substring of X , called the string represented by the node. Note that the prefix trie of X is identical to the suffix trie of reverse of X and therefore suffix trie theories can also be applied to prefix trie.

With the prefix trie, testing whether a query W is an exact substring of X is equivalent to finding the node that represents W , which can be done in $O(|W|)$ time by matching each symbol in W to an edge, starting from the root. To allow mismatches, we can exhaustively traverse the trie and match W to each possible path. We will later show how to accelerate this search by using prefix information of W . Figure 1 gives an example of the prefix trie for 'GOOGOL'. The suffix array (SA) interval in each node is explained in Section 2.3.

2.2 Burrows–Wheeler transform

Let Σ be an alphabet. Symbol $\$$ is not present in Σ and is lexicographically smaller than all the symbols in Σ . A string $X = a_0a_1 \dots a_{n-1}$ is always ended with symbol $\$$ (i.e. $a_{n-1} = \$$) and this symbol only appears at the end. Let $X[i] = a_i, i = 0, 1, \dots, n-1$, be the i -th symbol of X , $X[i, j] = a_i \dots a_j$ a substring and $X_i = X[i, n-1]$ a suffix of X . Suffix array S of X is a permutation of the integers $0 \dots n-1$ such that $S(i)$ is the start position of the i -th smallest suffix. The BWT of X is defined as $B[i] = \$$ when $S(i) = 0$ and $B[i] = X[S(i)-1]$ otherwise. We also define the length of string X as $|X|$ and therefore $|X| = |B| = n$. Figure 2 gives an example on how to construct BWT and suffix array.

The algorithm shown in Figure 2 is quadratic in time and space. However, this is not necessary. In practice, we usually construct the suffix array first and then generate BWT. Most algorithms for constructing suffix array require at least $n \lceil \log_2 n \rceil$ bits of working space, which amounts to 12 GB for human genome. Recently, Hon *et al.* (2007) gave a new algorithm that uses n bits of working space and only requires <1 GB memory at peak time for constructing the BWT of human genome. This algorithm is implemented in

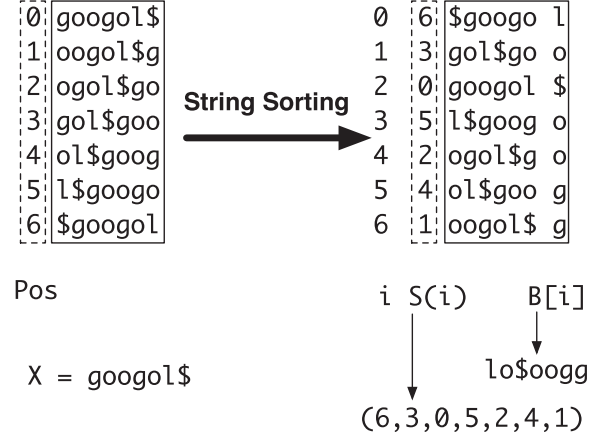


Fig. 2. Constructing suffix array and BWT string for $X = \text{googol\$}$. String X is circulated to generate seven strings, which are then lexicographically sorted. After sorting, the positions of the first symbols form the suffix array (6, 3, 0, 5, 2, 4, 1) and the concatenation of the last symbols of the circulated strings gives the BWT string lo\$ooogg.

BWT-SW (Lam *et al.*, 2008). We adapted its source code to make it work with BWA.

2.3 Suffix array interval and sequence alignment

If string W is a substring of X , the position of each occurrence of W in X will occur in an interval in the suffix array. This is because all the suffixes that have W as prefix are sorted together. Based on this observation, we define:

$$\underline{R}(W) = \min\{k : W \text{ is the prefix of } X_{S(k)}\} \quad (1)$$

$$\bar{R}(W) = \max\{k : W \text{ is the prefix of } X_{S(k)}\} \quad (2)$$

In particular, if W is an empty string, $\underline{R}(W) = 1$ and $\bar{R}(W) = n-1$. The interval $[\underline{R}(W), \bar{R}(W)]$ is called the *SA interval* of W and the set of positions of all occurrences of W in X is $\{S(k) : \underline{R}(W) \leq k \leq \bar{R}(W)\}$. For example in Figure 2, the SA interval of string 'go' is [1, 2]. The suffix array values in this interval are 3 and 0 which give the positions of all the occurrences of 'go'.

Knowing the intervals in suffix array we can get the positions. Therefore, sequence alignment is equivalent to searching for the SA intervals of substrings of X that match the query. For the exact matching problem, we can find only one such interval; for the inexact matching problem, there may be many.

2.4 Exact matching: backward search

Let $C(a)$ be the number of symbols in $X[0, n-2]$ that are lexicographically smaller than $a \in \Sigma$ and $O(a, i)$ the number of occurrences of a in $B[0, i]$. Ferragina and Manzini (2000) proved that if W is a substring of X :

$$\underline{R}(aW) = C(a) + O(a, \underline{R}(W)) - 1 + 1 \quad (3)$$

$$\bar{R}(aW) = C(a) + O(a, \bar{R}(W)) \quad (4)$$

and that $\underline{R}(aW) \leq \bar{R}(aW)$ if and only if aW is a substring of X . This result makes it possible to test whether W is a substring of X and to count the occurrences of W in $O(|W|)$ time by iteratively calculating $\underline{R}$ and $\bar{R}$ from the end of W . This procedure is called *backward search*.

It is important to note that Equations (3) and (4) actually realize the top-down traversal on the prefix trie of X given that we can calculate the SA interval of a child node in constant time if we know the interval of its parent. In this sense, backward search is equivalent to exact string matching on the prefix trie, but without explicitly putting the trie in the memory.

Precalculation:

Calculate BWT string B for reference string X
 Calculate array $C(\cdot)$ and $O(\cdot, \cdot)$ from B
 Calculate BWT string B' for the reverse reference
 Calculate array $O'(\cdot, \cdot)$ from B'

Procedures:

INEXACTSEARCH(W, z)
 CALCULATED(W)
 return INEXRECUR($W, |W| - 1, z, 1, |X| - 1$)

CALCULATED(W)

$k \leftarrow 1$
 $l \leftarrow |X| - 1$
 $z \leftarrow 0$
for $i = 0$ **to** $|W| - 1$ **do**
 $k \leftarrow C(W[i]) + O'(W[i], k - 1) + 1$
 $l \leftarrow C(W[i]) + O(W[i], l)$
if $k > l$ **then**
 $k \leftarrow 1$
 $l \leftarrow |X| - 1$
 $z \leftarrow z + 1$
 $D(i) \leftarrow z$

INEXRECUR(W, i, z, k, l)

if $z < D(i)$ **then**
 return $\emptyset$
if $i < 0$ **then**
 return $\{[k, l]\}$
 $I \leftarrow \emptyset$
 * $I \leftarrow I \cup \text{INEXRECUR}(W, i - 1, z - 1, k, l)$
for each $b \in \{A, C, G, T\}$ **do**
 $k \leftarrow C(b) + O(b, k - 1) + 1$
 $l \leftarrow C(b) + O(b, l)$
if $k \leq l$ **then**
 * $I \leftarrow I \cup \text{INEXRECUR}(W, i, z - 1, k, l)$
if $b = W[i]$ **then**
 $I \leftarrow I \cup \text{INEXRECUR}(W, i - 1, z, k, l)$
else
 $I \leftarrow I \cup \text{INEXRECUR}(W, i - 1, z - 1, k, l)$
return I

Fig. 3. Algorithm for inexact search of SA intervals of substrings that match W . Reference X is \$ terminated, while W is A/C/G/T terminated. Procedure INEXACTSEARCH(W, z) returns the SA intervals of substrings that match W with no more than z differences (mismatches or gaps); INEXRECUR(W, i, z, k, l) recursively calculates the SA intervals of substrings that match $W[0, i]$ with no more than z differences on the condition that suffix W_{i+1} matches interval $[k, l]$. Lines started with asterisk are for insertions to and deletions from X , respectively. $D(i)$ is the lower bound of the number of differences in string $W[0, i]$.

2.5 Inexact matching: bounded traversal/backtracking

Figure 3 gives a recursive algorithm to search for the SA intervals of substrings of X that match the query string W with no more than z differences (mismatches or gaps). Essentially, this algorithm uses backward search to sample distinct substrings from the genome. This process is bounded by the $D(\cdot)$ array where $D(i)$ is the lower bound of the number of differences in $W[0, i]$. The better the D is estimated, the smaller the search space and the more efficient the algorithm is. A naive bound is achieved by setting $D(i) = 0$

for all i , but the resulting algorithm is clearly exponential in the number of differences and would be less efficient.

CALCULATED(W)
 $z \leftarrow 0$
 $j \leftarrow 0$
for $i = 0$ **to** $|W| - 1$ **do**
if $W[j, i]$ is not a substring of X **then**
 $z \leftarrow z + 1$
 $j \leftarrow i + 1$
 $D(i) \leftarrow z$

Fig. 4. Equivalent algorithm to calculate $D(i)$.

The CALCULATED procedure in Figure 3 gives a better, though not optimal, bound. It is conceptually equivalent to the one described in Figure 4, which is simpler to understand. We use the BWT of the reverse (not complemented) reference sequence to test if a substring of W is also a substring of X . Note that to do this test with BWT string B alone would make CALCULATED an $O(|W|^2)$ procedure, rather than $O(|W|)$ as is described in Figure 3.

To understand the role of D , we come back to the example of searching for $W = \text{LOL}$ in $X = \text{GOOGOL\$}$ (Fig. 1). If we set $D(i) = 0$ for all i and disallow gaps (removing the two star lines in the algorithm), the call graph of INEXRECUR, which is a tree, effectively mimics the search route shown as the dashed line in Figure 1. However, with CALCULATED, we know that $D(0) = 0$ and $D(1) = D(2) = 1$. We can then avoid descending into the 'G' and 'O' subtrees in the prefix trie to get a much smaller search space.

The algorithm in Figure 3 guarantees to find all the intervals allowing maximum z differences. It is complete in theory, but in practice, we also made various modifications. First, we pay different penalties for mismatches, gap opens and gap extensions, which is more realistic to biological data. Second, we use a heap-like data structure to keep partial hits rather than using recursion. The heap-like structure is prioritized on the alignment score of the partial hits to make BWA always find the best intervals first. The reverse complemented read sequence is processed at the same time. Note that the recursion described in Figure 3 effectively mimics a depth-first search (DFS) on the prefix trie, while BWA implements a breadth-first search (BFS) using this heap-like data structure. Third, we adopt an iterative strategy: if the top interval is repetitive, we do not search for suboptimal intervals by default; if the top interval is unique and has z difference, we only search for hits with up to $z + 1$ differences. This iterative strategy accelerates BWA while retaining the ability to generate mapping quality. However, this also makes BWA's speed sensitive to the mismatch rate between the reads and the reference because finding hits with more differences is usually slower. Fourth, we allow to set a limit on the maximum allowed differences in the first few tens of base pairs on a read, which we call the *seed* sequence. Given 70 bp simulated reads, alignment with maximum two differences in the 32 bp seed is $2.5\times$ faster than without seeding. The alignment error rate, which is the fraction of wrong alignments out of confident mappings in simulation (see also Section 3.2), only increases from 0.08% to 0.11%. Seeding is less effective for shorter reads.

2.6 Reducing memory

The algorithm described above needs to load the occurrence array O and the suffix array S in the memory. Holding the full O and S arrays requires huge memory. Fortunately, we can reduce the memory by only storing a small fraction of the O and S arrays, and calculating the rest on the fly. BWT-SW (Lam *et al.*, 2008) and Bowtie (Langmead *et al.*, 2009) use a similar strategy which was first introduced by Ferragina and Manzini (2000).

Given a genome of size n , the occurrence array $O(\cdot, \cdot)$ requires $4n \lceil \log_2 n \rceil$ bits as each integer takes $\lceil \log_2 n \rceil$ bits and there are $4n$ of them in the array. In practice, we store in memory $O(\cdot, k)$ for k that is a factor of 128 and calculate the rest of elements using the BWT string B . When we use two bits to represent a nucleotide, B takes $2n$ bits. The memory for backward search is

thus $2n + n\lceil \log_2 n \rceil / 32$ bits. As we also need to store the BWT of the reverse genome to calculate the bound, the memory required for calculating intervals is doubled, or about 2.3 GB for a 3 Gb genome.

Enumerating the position of each occurrence requires the suffix array S . If we put the entire S in memory, it would use $n\lceil \log_2 n \rceil$ bits. However, it is also possible to reconstruct the entire S when knowing part of it. In fact, S and inverse compressed suffix array (inverse CSA) Ψ^{-1} (Grossi and Vitter, 2000) satisfy:

$$S(k) = S((\Psi^{-1})^{(j)}(k)) + j \quad (5)$$

where $(\Psi^{-1})^{(j)}$ denotes repeatedly applying the transform Ψ^{-1} for j times. The inverse CSA Ψ^{-1} can be calculated with the occurrence array O :

$$\Psi^{-1}(i) = C(B[i]) + O(B[i], i) \quad (6)$$

In BWA, we only store in memory $S(k)$ for k that can be divided by 32. For k that is not a factor of 32, we repeatedly apply Ψ^{-1} until for some j , $(\Psi^{-1})^{(j)}(k)$ is a factor of 32 and then $S((\Psi^{-1})^{(j)}(k))$ can be looked up and $S(k)$ can be calculated with Equation (5).

In all, the alignment procedure uses $4n + n\lceil \log_2 n \rceil / 8$ bits, or n bytes for genomes < 4 Gb. This includes the memory for the BWT string, partial occurrence array and partial suffix array for both original and the reversed genome. Additionally, a few hundred megabyte of memory is required for heap, cache and other data structures.

2.7 Other practical concerns for Illumina reads

2.7.1 Ambiguous bases Non-A/C/G/T bases on reads are simply treated as mismatches, which is implicit in the algorithm (Fig. 3). Non-A/C/G/T bases on the reference genome are converted to random nucleotides. Doing so may lead to false hits to regions full of ambiguous bases. Fortunately, the chance that this may happen is very small given relatively long reads. We tried 2 million 32 bp reads and did not see any reads mapped to poly-N regions by chance.

2.7.2 Paired-end mapping BWA supports paired-end mapping. It first finds the positions of all the good hits, sorts them according to the chromosomal coordinates and then does a linear scan through all the potential hits to pair the two ends. Calculating all the chromosomal coordinates requires to look up the suffix array frequently. This pairing process is time consuming as generating the full suffix array on the fly with the method described above is expensive. To accelerate pairing, we cache large intervals. This strategy halves the time spent on pairing.

In pairing, BWA processes 256K read pairs in a batch. In each batch, BWA loads the full BWA index into memory, generates the chromosomal coordinate for each occurrence, estimates the insert size distribution from read pairs with both ends mapped with mapping quality higher than 20, and then pairs them. After that, BWA clears the BWT index from the memory, loads the 2 bit encoded reference sequence and performs Smith–Waterman alignment for unmapped reads whose mates can be reliably aligned. Smith–Waterman alignment rescues some reads with excessive differences.

2.7.3 Determining the allowed maximum number of differences Given a read of length m , BWA only tolerates a hit with at most k differences (mismatches or gaps), where k is chosen such that $< 4\%$ of m -long reads with 2% uniform base error rate may contain differences more than k . With this configuration, for 15–37 bp reads, k equals 2; for 38–63 bp, $k = 3$; for 64–92 bp, $k = 4$; for 93–123 bp, $k = 5$; and for 124–156 bp reads, $k = 6$.

2.7.4 Generating mapping quality scores For each alignment, BWA calculates a mapping quality score, which is the Phred-scaled probability of the alignment being incorrect. The algorithm is similar to MAQ's except that in BWA we assume the true hit can always be found. We made this modification because we are aware that MAQ's formula overestimates the probability of missing the true hit, which leads to underestimated mapping quality. Simulation reveals that BWA may overestimate mapping quality due

to this modification, but the deviation is relatively small. For example, BWA wrongly aligns 11 reads out of 1 569 108 simulated 70 bp reads mapped with mapping quality 60. The error rate 7×10^{-6} ($= 11 / 1\,569\,108$) for these Q60 mappings is higher than the theoretical expectation 10^{-6} .

2.8 Mapping SOLiD reads

For SOLiD reads, BWA converts the reference genome to dinucleotide 'color' sequence and builds the BWT index for the color genome. Reads are mapped in the color space where the reverse complement of a sequence is the same as the reverse, because the complement of a color is itself. For SOLiD paired-end mapping, a read pair is said to be in the correct orientation if either of the two scenarios is true: (i) both ends mapped to the forward strand of the genome with the R3 read having smaller coordinate; and (ii) both ends mapped to the reverse strand of the genome with the F3 read having smaller coordinate. Smith–Waterman alignment is also done in the color space.

After the alignment, BWA decodes the color read sequences to the nucleotide sequences using dynamic programming. Given a nucleotide reference subsequence $b_1 b_2 \dots b_{l+1}$ and a color read sequence $c_1 c_2 \dots c_l$ mapped to the subsequence, BWA infers a nucleotide sequence $\hat{b}_1 \hat{b}_2 \dots \hat{b}_{l+1}$ such that it minimizes the following objective function:

$$\sum_{i=1}^{l+1} q' \cdot (1 - \delta_{b_i, b_i}) + \sum_{i=1}^l q_i \cdot [1 - \delta_{c_i, g(\hat{b}_i, \hat{b}_{i+1})}]$$

where q' is the Phred-scaled probability of a mutation, q_i is the Phred quality of color c_i and function $g(b, b') = g(b', b)$ gives the color corresponding to the two adjacent nucleotides b and b' . Essentially, we pay a penalty q' if $b_i \neq \hat{b}_i$ and a penalty q_i if $c_i \neq g(\hat{b}_i, \hat{b}_{i+1})$.

This optimization can be done by dynamic programming because the best decoding beyond position i only depends on the choice of $\hat{b}_i$. Let $f_i(\hat{b}_i)$ be the best decoding score up to i . The iteration equations are

$$f_1(\hat{b}_1) = q' \cdot (1 - \delta_{\hat{b}_1, b_1})$$

$$f_{i+1}(\hat{b}_{i+1}) = \min_{\hat{b}_i} \left\{ f_i(\hat{b}_i) + q' \cdot (1 - \delta_{\hat{b}_{i+1}, b_{i+1}}) + q_i \cdot [1 - \delta_{c_i, g(\hat{b}_i, \hat{b}_{i+1})}] \right\}$$

BWA approximates base qualities as follows. Let $\hat{c}_i = g(\hat{b}_i, \hat{b}_{i+1})$. The i -th base quality $\hat{q}_i$, $i = 2 \dots l$, is calculated as:

$$\hat{q}_i = \begin{cases} q_{i-1} + q_i & \text{if } c_{i-1} = \hat{c}_{i-1} \text{ and } c_i = \hat{c}_i \\ q_{i-1} - q_i & \text{if } c_{i-1} = \hat{c}_{i-1} \text{ but } c_i \neq \hat{c}_i \\ q_i - q_{i-1} & \text{if } c_i = \hat{c}_i \text{ but } c_{i-1} \neq \hat{c}_{i-1} \\ 0 & \text{otherwise} \end{cases}$$

BWA outputs the sequence $\hat{b}_2 \dots \hat{b}_l$ and the quality $\hat{q}_2 \dots \hat{q}_l$ as the final result for SOLiD mapping.

3 RESULTS

3.1 Implementation

We implemented BWA to do short read alignment based on the BWT of the reference genome. It performs gapped alignment for single-end reads, supports paired-end mapping, generates mapping quality and gives multiple hits if required. The default output alignment format is SAM (Sequence Alignment/Map format). Users can use SAMtools (<http://samtools.sourceforge.net>) to extract alignments in a region, merge/sort alignments, get single nucleotide polymorphism (SNP) and indel calls and visualize the alignment.

BWA is distributed under the GNU General Public License (GPL). Documentations and source code are freely available at the MAQ web site: <http://maq.sourceforge.net>.

3.2 Evaluated programs

To evaluate the performance of BWA, we tested additional three alignment programs: MAQ (Li *et al.*, 2008a), SOAPv2 (<http://soap.genomics.org.cn>) and Bowtie (Langmead *et al.*, 2009). MAQ indexes reads with a hash table and scans through the genome. It is the software package we developed previously for large-scale read mapping. SOAPv2 and Bowtie are the other two BWT-based short read aligners that we are aware of. The latest SOAP-2.1.7 (Li *et al.*, unpublished data) uses 2way-BWT (Lam *et al.*, unpublished data) for alignment. It tolerates more mismatches beyond the 35 bp seed sequence and supports gapped alignment limited to one gap open. Bowtie (version 0.9.9.2) deploys a similar algorithm to BWA. Nonetheless, it does not reduce the search space by bounding the search with $D(i)$, but by cleverly doing the alignment for both original and reverse read sequences to bypass unnecessary searches towards the root of the prefix trie. By default, Bowtie performs a DFS on the prefix trie and stops when the first qualified hit is found. Thus, it may miss the best inexact hit even if its seeding strategy is disabled. It is possible to make Bowtie perform a BFS by applying ‘-best’ at the command line, but this makes Bowtie slower. Bowtie does not support gapped alignment at the moment.

All the four programs, including BWA, randomly place a repetitive read across the multiple equally best positions. As we are mainly interested in confident mappings in practice, we need to rule out repetitive hits. SOAPv2 gives the number of equally best hits of a read. Only unique mappings are retained. We also ask SOAPv2 to limit the possible gap size to at most 3 bp. We run Bowtie with the command-line option ‘-best -k 2’, which renders Bowtie to output the top two hits of a read. We discard a read alignment if the second best hit contains the same number of mismatches as the best hit. MAQ and BWA generate mapping qualities. We use mapping quality threshold 1 for MAQ and 10 for BWA to determine confident mappings. We use different thresholds because we know that MAQ’s mapping quality is underestimated, while BWA’s is overestimated.

3.3 Evaluation on simulated data

We simulated reads from the human genome using the wgsim program that is included in the SAMtools package and ran the four programs to map the reads back to the human genome. As we know the exact coordinate of each read, we are able to calculate the alignment error rate.

Table 1 shows that BWA and MAQ achieve similar alignment accuracy. BWA is more accurate than Bowtie and SOAPv2 in terms of both the fraction of confidently mapped reads and the error rate of confident mappings. Note that SOAP-2.1.7 is optimized for reads longer than 35 bp. For the 32 bp reads, SOAP-2.0.1 outperforms the latest version.

On speed, SOAPv2 is the fastest and actually it would be 30–80% faster for paired-end mapping if gapped alignment was disabled. Bowtie with the default option (data not shown) is several times faster than the current setting ‘-best -k 2’ on single-end mapping. However, the speed is gained at a great cost of accuracy. For example, with the default option, Bowtie can map the two million single-end 32 bp reads in 151 s, but 6.4% of confident mappings are wrong. This high alignment error rate may complicate the detection of structural variations and potentially affect SNP accuracy. Between

Table 1. Evaluation on simulated data

Program	Single-end			Paired-end		
	Time (s)	Conf (%)	Err (%)	Time (s)	Conf (%)	Err (%)
Bowtie-32	1271	79.0	0.76	1391	85.7	0.57
BWA-32	823	80.6	0.30	1224	89.6	0.32
MAQ-32	19797	81.0	0.14	21589	87.2	0.07
SOAP2-32	256	78.6	1.16	1909	86.8	0.78
Bowtie-70	1726	86.3	0.20	1580	90.7	0.43
BWA-70	1599	90.7	0.12	1619	96.2	0.11
MAQ-70	17928	91.0	0.13	19046	94.6	0.05
SOAP2-70	317	90.3	0.39	708	94.5	0.34
bowtie-125	1966	88.0	0.07	1701	91.0	0.37
BWA-125	3021	93.0	0.05	3059	97.6	0.04
MAQ-125	17506	92.7	0.08	19388	96.3	0.02
SOAP2-125	555	91.5	0.17	1187	90.8	0.14

One million pairs of 32, 70 and 125 bp reads, respectively, were simulated from the human genome with 0.09% SNP mutation rate, 0.01% indel mutation rate and 2% uniform sequencing base error rate. The insert size of 32 bp reads is drawn from a normal distribution $N(170, 25)$, and of 70 and 125 bp reads from $N(500, 50)$. CPU time in seconds on a single core of a 2.5 GHz Xeon E5420 processor (Time), percent confidently mapped reads (Conf) and percent erroneous alignments out of confident mappings (Err) are shown in the table.

BWA and MAQ, BWA is 6–18 $\times$ faster, depending on the read length. MAQ’s speed is not affected by read length because internally it treats all reads as 128 bp. It is possible to accelerate BWA by not checking suboptimal hits similar to what Bowtie and SOAPv2 are doing. However, calculating mapping quality would be impossible in this case and we believe generating proper mapping quality is useful to various downstream analyses such as the detection of structural variations.

On memory, SOAPv2 uses 5.4 GB. Both Bowtie and BWA uses 2.3 GB for single-end mapping and about 3 GB for paired-end, larger than MAQ’s memory footprint 1 GB. However, the memory usage of all the three BWT-based aligners is independent of the number of reads to be aligned, while MAQ’s is linear in it. In addition, all BWT-based aligners support multi-threading, which reduces the memory per CPU core on a multi-core computer. On modern computer servers, memory is not a practical concern with the BWT-based aligners.

3.4 Evaluation on real data

To assess the performance on real data, we downloaded about 12.2 million pairs of 51 bp reads from European Read Archive (AC:ERR000589). These reads were produced by Illumina for NA12750, a male included in the 1000 Genomes Project (<http://www.1000genomes.org>). Reads were mapped to the human genome NCBI build 36. Table 2 shows that almost all confident mappings from MAQ and BWA exist in consistent pairs although MAQ gives fewer confident alignments. A slower mode of BWA (no seeding; searching for suboptimal hits even if the top hit is a repeat) did even better. In that mode, BWA confidently mapped 89.2% of all reads in 6.3 hours with 99.2% of confident mappings in consistent pairs.

In this experiment, SOAPv2 would be twice as fast with both percent confident mapping (Conf) and percent paired (Paired)

Table 2. Evaluation on real data

Program	Time (h)	Conf (%)	Paired (%)
Bowtie	5.2	84.4	96.3
BWA	4.0	88.9	98.8
MAQ	94.9	86.1	98.7
SOAP2	3.4	88.3	97.5

The 12.2 million read pairs were mapped to the human genome. CPU time in hours on a single core of a 2.5 GHz Xeon E5420 processor (Time), percent confidently mapped reads (Conf) and percent confident mappings with the mates mapped in the correct orientation and within 300 bp (Paired), are shown in the table.

dropping by 1% if gapped alignment was disabled. In contrast, BWA is 1.4 times as fast when it performs ungapped alignment only. But even with BWT-based gapped alignment disabled, BWA is still able to recover many short indels with Smith–Waterman alignment given paired-end reads.

We also obtained the chicken genome sequence (version 2.1) and aligned these 12.2 million read pairs against a human–chicken hybrid reference sequence. The percent confident mappings is almost unchanged in comparison to the human-only alignment. As for the number of reads mapped to the chicken genome, Bowtie mapped 2640, BWA 2942, MAQ 3005 and SOAPv2 mapped 4531 reads to the wrong genome. If we consider that the chicken sequences take up one-quarter of the human–chicken hybrid reference, the alignment error rate for BWA is about 0.06% ($=2942 \times 4 / 12.2M / 0.889$). Note that such an estimate of the alignment error rate may be underestimated because wrongly aligned human reads tend to be related to repetitive sequences in human and to be mapped back to the human sequences. The estimate may also be overestimated due to the presence of highly conservative sequences and the incomplete assembly of human or misassembly of the chicken genome.

If we want fewer errors, the mapping quality generated by BWA and MAQ allows us to choose alignments of higher accuracy. If we increased the mapping quality threshold in determining a confident hit to 25 for BWA, 86.4% of reads could be aligned confidently with 1927 reads mapped to the chicken genome, outperforming Bowtie in terms of both percent confident mappings and the number of reads mapped to the wrong genome.

4 DISCUSSION

For short read alignment against the human reference genome, BWA is an order of magnitude faster than MAQ while achieving similar alignment accuracy. It supports gapped alignment for single-end reads, which is increasingly important when reads get longer and tend to contain indels. BWA outputs alignment in the SAM format to take the advantage of the downstream analyses implemented in SAMtools. BWA plus SAMtools provides most of functionality of the MAQ package with additional features.

In comparison to speed, memory and the number of mapped reads, alignment accuracy is much harder to evaluate on real data as we do not know the ground truth. In this article, we used three criteria for evaluating the accuracy of an aligner. The first criterion, which can only be evaluated with simulated data, is the combination of the number of confident mappings and the alignment error rate

out of the confident mappings. Note that the number of confident mappings alone may not be a good criterion: we can map more at the cost of accuracy. The second criterion, which is the combination of the number of aligned reads and the number of reads mapped in consistent pairs, works on real data on the assumption that the mating information from the experiment is correct and that structural variations are rare. Although this criterion is related to the way an aligner defines pair ‘consistency’, in our experience it is highly informative if the pairing parameters are set correctly. The third criterion is the fraction of reads mapped to the wrong reference sequence if we intentionally add reference sequences from a diverged species.

Although in theory BWA works with arbitrarily long reads, its performance is degraded on long reads especially when the sequencing error rate is high. Furthermore, BWA always requires the full read to be aligned, from the first base to the last one (i.e. global with respect to reads), but longer reads are more likely to be interrupted by structural variations or misassemblies in the reference genome, which will fail BWA. For long reads, a possibly better solution would be to divide the read into multiple short fragments, align the fragments separately with the algorithm described above and then join the partial alignments to get the full alignment of the read.

ACKNOWLEDGEMENTS

We are grateful to Mark DePristo and Jared Maguire from the Broad Institute for their suggestions on standardizing the criteria for evaluating alignment programs, and to the three anonymous reviewers whose comments helped us to improve the manuscript. We also thank the members of the Durbin research group for the comments on the initial draft.

Funding: Wellcome Trust/077192/Z/05/Z.

Conflict of Interest: none declared.

REFERENCES

- Burrows,M. and Wheeler,D.J. (1994) A block-sorting lossless data compression algorithm. *Technical report 124*, Palo Alto, CA, Digital Equipment Corporation.
- Campagna,D. *et al.* (2009) PASS: a program to align short sequences. *Bioinformatics*, **25**, 967–968.
- Eaves,H.L. and Gao,Y. (2009) MOM: maximum oligonucleotide mapping. *Bioinformatics*, **25**, 969–970.
- Ferragina,P. and Manzini,G. (2000) Opportunistic data structures with applications. In *Proceedings of the 41st Symposium on Foundations of Computer Science (FOCS 2000)*, IEEE Computer Society, pp. 390–398.
- Grossi,R. and Vitter,J.S. (2000) Compressed suffix arrays and suffix trees with applications to text indexing and string matching. In *Proceedings on 32nd Annual ACM Symposium on Theory of Computing (STOC 2000)*, ACM, pp. 397–406.
- Hon,W.-K. *et al.* (2007) A space and time efficient algorithm for constructing compressed suffix arrays. *Algorithmica*, **48**, 23–36.
- Jiang,H. and Wong,W.H. (2008) SeqMap: mapping massive amount of oligonucleotides to the genome. *Bioinformatics*, **24**, 2395–2396.
- Jung Kim,Y. *et al.* (2009) ProbeMatch: a tool for aligning oligonucleotide sequences. *Bioinformatics*, **25**, 1424–1425.
- Lam,T.W. *et al.* (2008) Compressed indexing and local alignment of DNA. *Bioinformatics*, **24**, 791–797.
- Langmead,B. *et al.* (2009) Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biol.*, **10**, R25.
- Lin,H. *et al.* (2008) ZOOM! Zillions of oligos mapped. *Bioinformatics*, **24**, 2431–2437.
- Lippert,R.A. (2005) Space-efficient whole genome comparisons with Burrows–Wheeler transforms. *J. Comput. Biol.*, **12**, 407–415.

- Li,H. *et al.* (2008a) Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Res.*, **18**, 1851–1858.
- Li,R. *et al.* (2008b) SOAP: short oligonucleotide alignment program. *Bioinformatics*, **24**, 713–714.
- Malhis,N. *et al.* (2009) Slider–maximum use of probability information for alignment of short sequence reads and SNP detection. *Bioinformatics*, **25**, 6–13.
- Schatz,M. (2009) Cloudburst: highly sensitive read mapping with mapreduce. *Bioinformatics*, **25**, 1363–1369.
- Smith,A.D. *et al.* (2008) Using quality scores and longer reads improves accuracy of Solexa read mapping. *BMC Bioinformatics*, **9**, 128.